MANIPAL UNIVERSITY JAIPUR

Department of Computer Science and Engineering

A Project Report on

Measuring the Indic Tokenizer Tax: A Comparative Study of Sarvam AI's Tokenizers and Akshara-Aware Subword Segmentation for Indian Languages

Submitted in partial fulfilment of the requirements of the course

Natural Language Processing (NLP)

Submitted by

Nikhileshwar Jakka

Reg. No. 23FE10CII00213

B.Tech CSE (IoT & Intelligent Systems)

Under the guidance of

Heena Kalim

Department of Computer Science and Engineering

Manipal University Jaipur, Jaipur, Rajasthan, India

November 2026

Certificate

This is to certify that the project report entitled “Measuring the Indic Tokenizer Tax: A Comparative Study of Sarvam AI's Tokenizers and Akshara-Aware Subword Segmentation for Indian Languages”, submitted by Nikhileshwar Jakka (Reg. No. 23FE10CII00213) in partial fulfilment of the requirements of the course Natural Language Processing (NLP), is a bonafide record of the work carried out by the student under my supervision during the academic year 2026–27.

Heena Kalim
Department of Computer Science and Engineering
Manipal University Jaipur

Date: _____ Signature: _____

Declaration

I, Nikhileshwar Jakka (Reg. No. 23FE10CII00213), hereby declare that the project report entitled "Measuring the Indic Tokenizer Tax: A Comparative Study of Sarvam AI's Tokenizers and Akshara-Aware Subword Segmentation for Indian Languages" is my own work, carried out under the guidance of Heena Kalim. All results reported here were produced by code that I ran and that is publicly available. All sources have been acknowledged and cited. This work has not been submitted elsewhere for any other degree or course.

Nikhileshwar Jakka

Reg. No. 23FE10CII00213

Place: Jaipur Date: _____

Acknowledgement

I thank my guide, Heena Kalim, for advice and feedback throughout this project, and the Department of Computer Science and Engineering, Manipal University Jaipur, for the course that motivated it. I also thank the teams who released the open resources this study relies on: Meta AI for FLORES-200, AI4Bharat for Sangraha, IndicBERTv2 and IndicTrans2, Sarvam AI, Google, Meta, Alibaba Qwen, Mistral AI and OpenAI for public tokenizers, and the maintainers of SentencePiece, Hugging Face Tokenizers and the Noto fonts.

Abstract

Large language models read text through a tokenizer, and the same content can need many more tokens in an Indian language than in English. That raises the cost of every API call, shrinks the usable context and slows generation. This project measures this “tokenizer tax” for 11 Indian languages on the parallel FLORES-200 devtest set across 12 published tokenizers, with a focus on the three generations of Sarvam AI models. Exact UTF-8 byte spans are recovered for every token, which lets us measure how often tokenizers break aksharas, the orthographic syllables of Brahmic scripts. Sarvam-M turns out to reuse the Mistral-Small-3.1 vocabulary unchanged (131,072 identical entries), and Sarvam-30B/105B shares 91.3% of its vocabulary, with identical ids, with Gemma-3. Compared with Sarvam-1, Sarvam-30B lowers fertility on 5 Indo-Aryan languages but raises it on all four Dravidian languages, widening the Dravidian/Indo-Aryan fertility ratio from 1.05 to 1.32. Byte-level tokenizers such as Llama-3.1 split 91.4% of aksharas. Finally, the project proposes akshara-atomic BPE, which gives every frequent akshara its own base symbol. At a 64K vocabulary it cuts the Akshara Split Rate from 8.3% to 0.6% for a 5.1% increase in fertility.

Keywords: tokenization, Indian languages, byte-pair encoding, fertility, akshara, Sarvam AI

Table of Contents

Chapter 1: Introduction 9

1.1 Background 9

1.2 Problem statement 9

1.3 Objectives 9

1.4 Scope 10

Chapter 2: Literature Review 11

Chapter 3: Methodology and System Design 13

3.1 Data 13

3.2 Tokenizer zoo 13

3.3 Metrics 14

3.4 Akshara-atomic BPE 14

Chapter 4: Implementation 15

4.1 Tools and environment 15

4.2 Repository structure 15

4.3 Key algorithms 15

4.4 Verification 16

Chapter 5: Results and Discussion 17

5.1 Overall picture 17

5.2 RQ1: The Sarvam lineage 18

5.3 RQ2: Indo-Aryan vs Dravidian 19

5.4 RQ3: Broken aksharas 20

5.5 RQ4: Akshara-atomic BPE 21

5.6 Discussion 22

Chapter 6: Conclusion and Future Work 23

References 24

Appendix A: Sample tokenizations 26

Appendix B: Full results 26

List of Figures

Figure 1: Method pipeline: evaluation branch (top) and training branch (bottom). 13
Figure 2: Fertility on FLORES-200 devtest for every tokenizer and language († marks more than 5% [UNK] tokens). 17
Figure 3: Parity relative to English, averaged over Indo-Aryan and Dravidian languages. 18
Figure 4: Fertility of the three Sarvam tokenizers with 95% bootstrap CIs. 19
Figure 5: Words that fit in an 8,192-token context window. 20
Figure 6: Akshara Split Rate by tokenizer and language family. 20
Figure 7: Vocabulary composition by script. 21
Figure 8: Fertility and ASR against vocabulary size. 22
Figure 9: One parallel FLORES sentence in Hindi and Telugu under five tokenizers. Red bars mark token boundaries inside an akshara; grey hex boxes are byte fragments. 26

List of Tables

Table 1: Summary of prior work 11
Table 2: Tokenizers evaluated 13
Table 3: Repository map 15
Table 4: Sarvam-1 vs Sarvam-30B fertility (paired Wilcoxon; $r < 0$ means Sarvam-30B is higher) 19
Table 5: Our tokenizers, mean over 11 Indian languages 21
Table 6: Fertility (tokens per word) for every tokenizer and language 26
Table 7: Akshara Split Rate (%) for every tokenizer and Indian language 26

Chapter 1: Introduction

1.1 Background

A tokenizer turns text into the integer ids a language model consumes. Modern LLMs use subword tokenizers such as byte-pair encoding (BPE) and Unigram, learned mostly from English-heavy data. When a vocabulary has few entries for a script, text in that script breaks into many short pieces, sometimes single bytes. On FLORES-200, GPT-4o's o200k tokenizer needs 1.93× as many tokens for Telugu as for the same content in English, and Llama-3.1 needs 8.29×. Because APIs charge per token and context windows are counted in tokens, this premium falls directly on Indian users.

Indian scripts are abugidas. The natural writing unit is the akshara, an orthographic syllable built from consonants, viramas (vowel killers) and dependent vowel signs. A boundary inside an akshara leaves a vowel sign or virama as a token on its own, which no reader would treat as a unit.

1.2 Problem statement

Sarvam AI, a Bengaluru company, has released three generations of models with three different tokenizers: Sarvam-1 (October 2024), Sarvam-M (May 2025) and the open-weight Sarvam-30B and Sarvam-105B (March 2026). There is no independent, reproducible measurement of how these tokenizers differ, how they compare with global tokenizers, whether Dravidian languages pay more than Indo-Aryan ones, or how often aksharas are broken.

1.3 Objectives

1. **RQ1.** Trace how Sarvam's tokenizers changed across generations and compare them with general multilingual tokenizers.
2. **RQ2.** Test for a systematic cost gap between Indo-Aryan and Dravidian languages, and between scripts.
3. **RQ3.** Measure how often tokenizers break aksharas.
4. **RQ4.** Design akshara-atomic BPE and measure its effect on akshara splits and fertility across vocabulary sizes.

1.4 Scope

The study covers English and 11 Indian languages in 9 Brahmic scripts: seven Indo-Aryan (Hindi, Marathi, Bengali, Assamese, Gujarati, Punjabi, Odia) and four Dravidian (Telugu, Tamil, Kannada, Malayalam). All metrics are intrinsic and were computed on a CPU laptop. No language model was trained, and only tokenizer files were downloaded.

Chapter 2: Literature Review

Sennrich et al. introduced BPE for neural machine translation [1]. Kudo proposed the Unigram language model tokenizer with subword regularisation [2], and SentencePiece made both language-independent by operating on raw text [3]. Byte-level BPE (tiktoken [4], Llama 3 [5]) works on UTF-8 bytes, so it never produces unknown tokens but can split a character into fragments.

Rust et al. defined fertility and the proportion of continued words and showed that a poor tokenizer hurts monolingual performance of multilingual models [6]. Petrov et al. introduced tokenization parity on parallel text and found premiums of up to 15× [7]. Ahia et al. quantified how this becomes a cost disparity in commercial APIs [8]. Zouhar et al. related tokenization quality to Rényi entropy [9].

For Indian languages, IndicNLPSuite [10], IndicBERTv2 with IndicXTREME [11] and MuRIL [12] built Indic encoders with large Indic vocabularies; IndicTrans2 released the IN22 benchmark [13]; Sangraha, part of IndicLLMSuite, provides verified pre-training text [14]; FLORES-200 provides professionally translated parallel sentences [15], [16]. Recent work focuses on Indic tokenizers directly: Brahma et al. evaluate BPE and Unigram on 17 Indian languages [17], MorphTok adds morphology-aware pre-tokenization and a constrained BPE that keeps dependent vowel signs attached [18], and Rana et al. design a multilingual tokenizer with an Indic variant [19]. Sarvam AI documents its models in official blog posts [20]–[22].

Table 1: Summary of prior work

Work	Year	Focus	Relevance here
Sennrich et al. [1]	2016	BPE subwords for NMT	Base algorithm for most tokenizers studied
Kudo; Kudo & Richardson [2], [3]	2018	Unigram LM, SentencePiece	Training toolkit for our models
Rust et al. [6]	2021	Fertility, continued words	Core metrics
Petrov et al. [7]	2023	Parity across languages	Parity metric, fairness framing
Ahia et al. [8]	2023	API cost disparity	Cost multiplier interpretation
Zouhar et al. [9]	2023	Rényi efficiency	Additional intrinsic metric
Doddapaneni et al. [11]	2023	IndicBERTv2, IndicXTREME	Indic encoder baseline
Khanuja et al. [12]	2021	MuRIL	Indic encoder baseline
Khan et al. [14]	2024	Sangraha corpus	Training data for our tokenizers
Brahma et al. [17]	2025	Indic tokenizer evaluation	Closest evaluation study
Brahma et al. (MorphTok) [18]	2025	Constrained BPE for abugidas	Closest method to akshara-atomic BPE

The gap this project fills is a reproducible, statistically tested comparison of the Sarvam tokenizer lineage, an exact byte-level measurement of akshara splits for every tokenizer family, and an akshara-atomic BPE that removes such splits by construction.

Chapter 3: Methodology and System Design

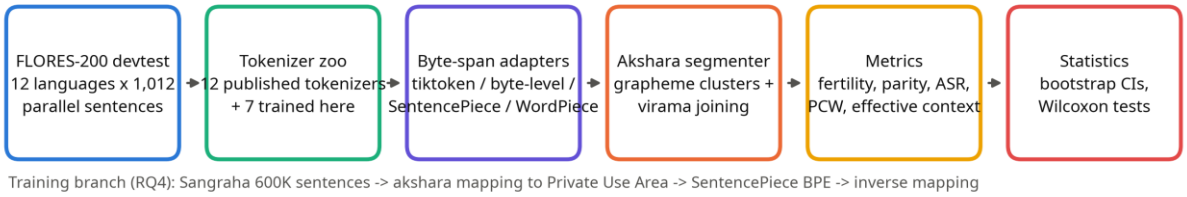


Figure 1: Method pipeline: evaluation branch (top) and training branch (bottom).

3.1 Data

Evaluation uses FLORES-200 devtest, 1,012 parallel sentences per language, from the official tarball. The IN22-Gen benchmark is gated on the Hugging Face Hub and was skipped. For training, 50,000 sentences per language (600,000 in total, plus 1,200 held out) were streamed from the verified split of Sangraha, deduplicated, and filtered so that no sentence shares a normalised 40-character substring with FLORES dev or devtest (2 sentences matched).

3.2 Tokenizer zoo

Table 2: Tokenizers evaluated

Tokenizer	Source	Algorithm	Vocab	Spans
Sarvam-1	sarvamai/sarvam-1	SentencePiece BPE+byte-fallback	68,096	bytes
Sarvam-M	sarvamai/sarvam-m	byte-level BPE	131,072	bytes
Sarvam-30B/105B	sarvamai/sarvam-30b	SentencePiece BPE+byte-fallback	262,144	bytes
GPT-4o (o200k)	o200k_base	tiktoken byte-level BPE	200,019	bytes
GPT-4 (cl100k)	cl100k_base	tiktoken byte-level BPE	100,277	bytes
Llama-3.1	unsloth/Llama-3.1-8B	byte-level BPE	128,256	bytes
Gemma-3	unsloth/gemma-3-1b-it	SentencePiece BPE+byte-fallback	262,145	bytes
Qwen2.5	Qwen/Qwen2.5-7B	byte-level BPE	151,665	bytes
MuRIL	google/muril-base-cased	WordPiece	197,258	offsets
IndicBERTv2	ai4bharat/IndicBERTv2-MLM-only	WordPiece	250,000	offsets
XLM-R	FacebookAI/xlm-roberta-base	SentencePiece Unigram	250,002	offsets
mBERT	google-bert/bert-base-multilingual-cased	WordPiece	119,547	offsets
Ours Akshara-BPE 16K	D:\PROJECTS\indic-tokenizer-study\models\ours-akshara-bpe-16k.model	SentencePiece bpe (akshara-atomic)	16,000	bytes
Ours Akshara-BPE 32K	D:\PROJECTS\indic-tokenizer-study\models\ours-akshara-bpe-32k.model	SentencePiece bpe (akshara-atomic)	32,000	bytes
Ours Akshara-BPE 64K	D:\PROJECTS\indic-tokenizer-study\models\ours-akshara-bpe-64k.model	SentencePiece bpe (akshara-atomic)	64,000	bytes
Ours BPE 16K	D:\PROJECTS\indic-tokenizer-study\models\ours-bpe-16k.model	SentencePiece bpe	16,000	bytes
Ours BPE 32K	D:\PROJECTS\indic-tokenizer-study\models\ours-bpe-32k.model	SentencePiece bpe	32,000	bytes
Ours BPE 64K	D:\PROJECTS\indic-tokenizer-study\models\ours-bpe-64k.model	SentencePiece bpe	64,000	bytes
Ours Unigram 64K	D:\PROJECTS\indic-tokenizer-study\models\ours-unigram-64k.model	SentencePiece unigram	64,000	bytes

Llama-3.1 and Gemma-3 were loaded from public mirrors. The Llama-3.1 mirror's 128,000 token-to-id pairs match Meta's official llama-models file exactly, and the Gemma-3 mirror's files have the same git blob hashes as Google's repository. Sarvam-30B and Sarvam-105B have byte-identical vocabularies and merges, so they are reported as one row.

3.3 Metrics

Words are obtained by whitespace splitting with punctuation removed, using one rule for every tokenizer.

- **Fertility** = total tokens / total words.
- **Parity** = tokens for the language / tokens for the parallel English sentences (the API cost multiplier).
- **Akshara Split Rate (ASR)** = % of aksharas that contain an internal token boundary.
- **Intra-character split rate** = % of codepoints whose UTF-8 bytes are split across tokens.
- **Proportion of continued words (PCW)**, **words per 8,192 tokens**, **[UNK] rate** and **Rényi efficiency** ($\alpha = 2.5$).

Uncertainty is reported as 95% bootstrap confidence intervals (1,000 resamples of sentences). Comparisons use paired Wilcoxon signed-rank tests over FLORES sentences with Holm correction, and the rank-biserial correlation r as effect size.

3.4 Akshara-atomic BPE

Every Indic akshara in the training corpus is counted. The most frequent $\min(n_2, V/2)$ types, where $n_2 = 22{,}221$ types occur at least twice, are mapped to codepoints in the Supplementary Private Use Area-A (U+F0000 onwards). SentencePiece BPE is trained on the mapped text, and the encode/decode wrappers apply and reverse the mapping. Merges can combine aksharas but can never split a mapped one.

Chapter 4: Implementation

4.1 Tools and environment

Python 3.11 in a virtual environment on a Windows 11 laptop (Intel Core i5-11320H, 16 GB RAM, no GPU). Libraries: Hugging Face transformers and tokenizers, tiktoken, SentencePiece, regex, pandas, SciPy, matplotlib, python-docx and python-pptx. The paper was typeset with tectonic (XeLaTeX). Exact versions are logged in results/run_metadata.json.

4.2 Repository structure

Table 3: Repository map

Path	Purpose
src/data.py	FLORES loader; Sangraha streaming, filtering, deduplication, overlap removal
src/tokenizers_zoo.py	Common interface and byte-span adapters for all tokenizer families
src/akshara.py	Akshara segmenter with per-script rules
src/metrics.py	Sentence-level and aggregated metrics
src/stats.py	Bootstrap CIs, Wilcoxon tests, Holm correction
src/vocab_analysis.py	Vocabulary script composition and lineage comparison
src/train_tokenizers.py	Standard, akshara-atomic and Unigram tokenizers
src/plots.py	All figures (300 dpi PNG + PDF)
src/report_numbers.py	Every number quoted in paper, report and slides
tests/	Akshara, metric and round-trip tests (pytest)

4.3 Key algorithms

Akshara joining: after grapheme clustering, a cluster that ends in a virama joins the next one when that cluster starts with a consonant of the same script (Tamil and Malayalam chillu rules aside):

```
for cl, start, end in _base_clusters(text):
    if prev and is_consonant(cl[0]) and script_of(cl[0]) == prev.script \
            and prev.rule.join_virama:
        out[-1][1] = end          # extend the previous akshara
    else:
        out.append([start, end])
    prev = dead_consonant(cl)   # ends in virama (+ZWJ/ZWNJ)?
```

Byte spans for byte-level BPE map each token back through the GPT-2 byte-to-unicode table, then assert that the pieces reconstruct the input exactly:

```
pieces = [bytes(BYTE_DECODER[c] for c in tok) for tok in enc.tokens]
assert b''.join(pieces) == normalised_text.encode('utf-8')
```

The akshara mapping replaces each mapped akshara with one private-use codepoint before SentencePiece sees it:

```
    def map(self, text):
        return ''.join(self.fwd.get(text[s:e], text[s:e]) for s, e in akshara_spans(text))
```

4.4 Verification

The test suite covers 30 hand-checked akshara segmentations across all nine scripts, the metric helpers, and a round-trip test showing decode(encode(x)) = x on 1,000 held-out sentences for all seven trained tokenizers. Span validation succeeded for at least 12,119 of 12,144 sentences for every published tokenizer.

Chapter 5: Results and Discussion

5.1 Overall picture

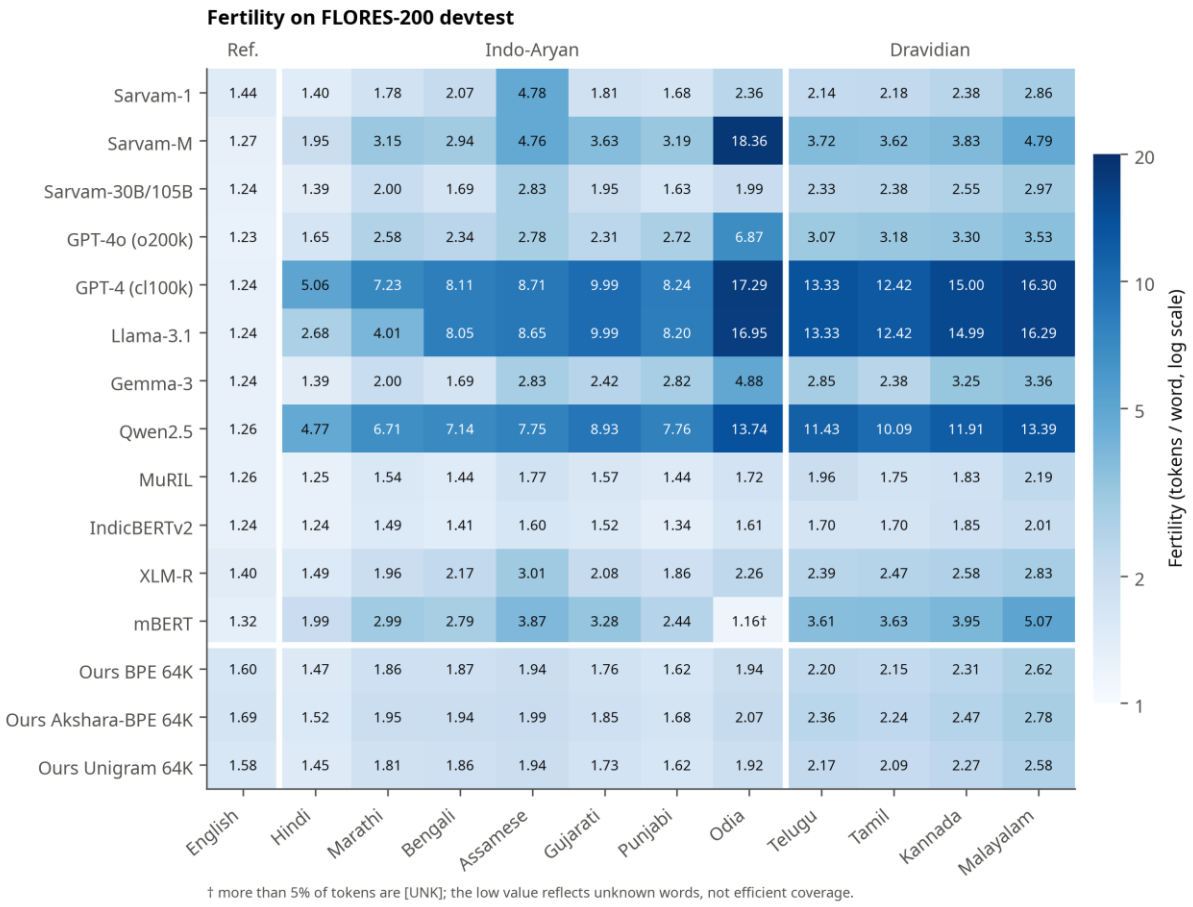


Figure 2: Fertility on FLORES-200 devtest for every tokenizer and language († marks more than 5% [UNK] tokens).

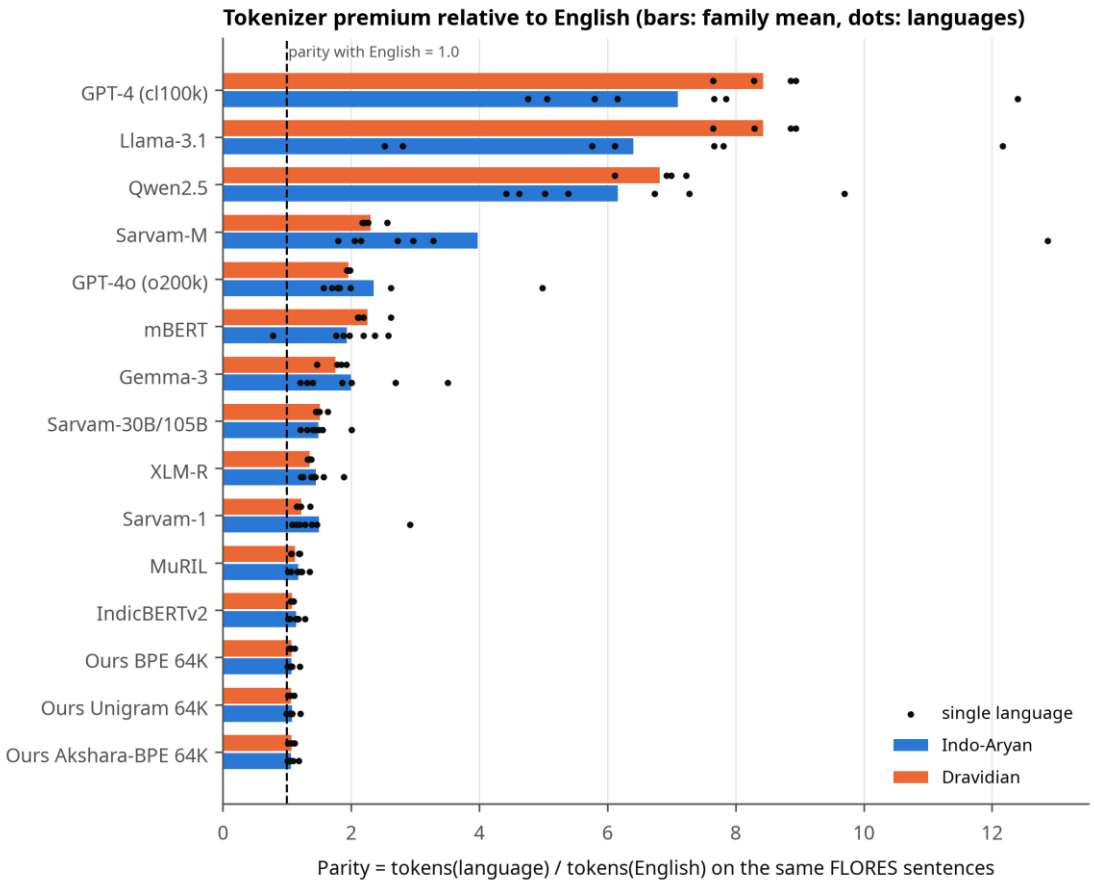


Figure 3: Parity relative to English, averaged over Indo-Aryan and Dravidian languages.

5.2 RQ1: The Sarvam lineage

Sarvam-1 devotes 73.0% of its 68K vocabulary to Indian scripts and reaches a mean Indic fertility of 2.31. Sarvam-M uses the Mistral-Small-3.1 vocabulary unchanged (all 131,072 entries identical), so its fertility rises to 4.90, reaching 18.36 on Odia. The Sarvam-30B/105B vocabulary shares 239,314 entries (91.3%) with Gemma-3 at identical ids. Its 22,828 new entries replace mainly CJK, Latin and Thai tokens with Gurmukhi, Odia, Kannada, Gujarati, Telugu and Malayalam tokens. Devanagari, Bengali and Tamil received none, so Sarvam-30B tokenizes Hindi, Marathi, Bengali, Assamese and Tamil exactly like Gemma-3.

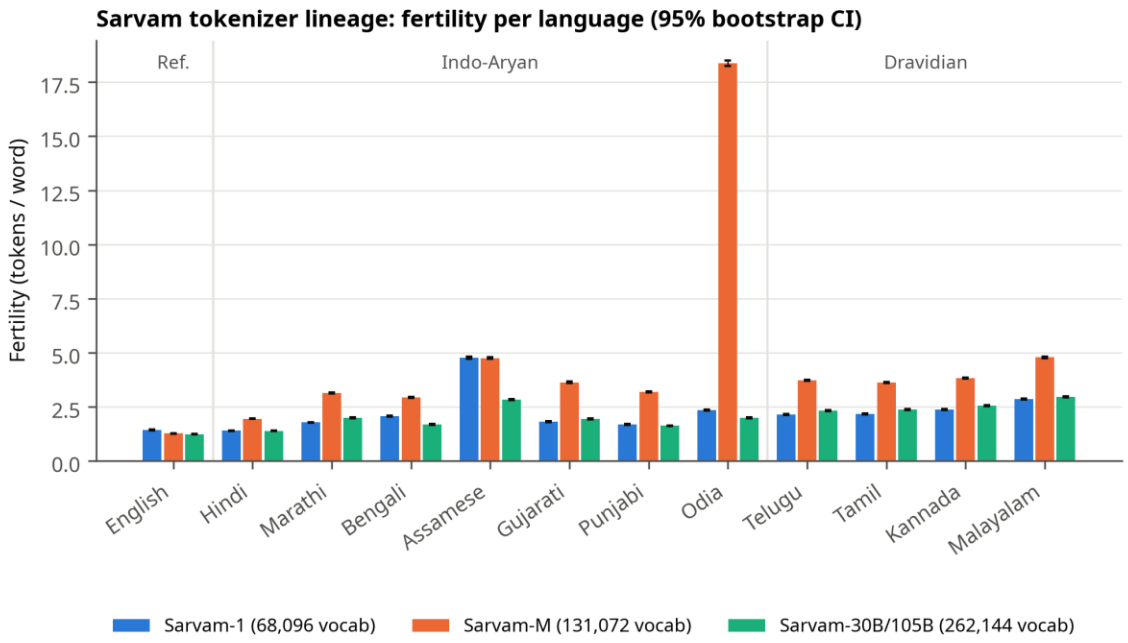


Figure 4: Fertility of the three Sarvam tokenizers with 95% bootstrap CIs.

Table 4: Sarvam-1 vs Sarvam-30B fertility (paired Wilcoxon; r < 0 means Sarvam-30B is higher)

Language	Sarvam-1	Sarvam-30B	p (Holm)	r
Hindi	1.40	1.39	<0.001	0.16
Marathi	1.78	2.00	<0.001	-0.96
Bengali	2.07	1.69	<0.001	0.94
Assamese	4.78	2.83	<0.001	1.00
Gujarati	1.81	1.95	<0.001	-0.79
Punjabi	1.68	1.63	<0.001	0.23
Odia	2.36	1.99	<0.001	0.99
Telugu	2.14	2.33	<0.001	-0.71
Tamil	2.18	2.38	<0.001	-0.88
Kannada	2.38	2.55	<0.001	-0.69
Malayalam	2.86	2.97	<0.001	-0.42

Sarvam-30B is better on 5 languages and worse on 6, including all four Dravidian ones; 11 of the 11 differences are significant.

5.3 RQ2: Indo-Aryan vs Dravidian

Dravidian fertility is significantly higher for 18 of 19 tokenizers. The ratio grows from 1.05 for Sarvam-1 to 1.32 for Sarvam-30B. Part of the gap is morphological (agglutinative words are longer and fewer). Parity, which controls for content, shows Dravidian text costing more in only 10 tokenizers, mostly byte-level ones with few Indic merges (Llama-3.1 ratio 1.32). Script coverage dominates family: Odia is the most expensive language for GPT-4o (parity 4.98) and Sarvam-M (12.87).

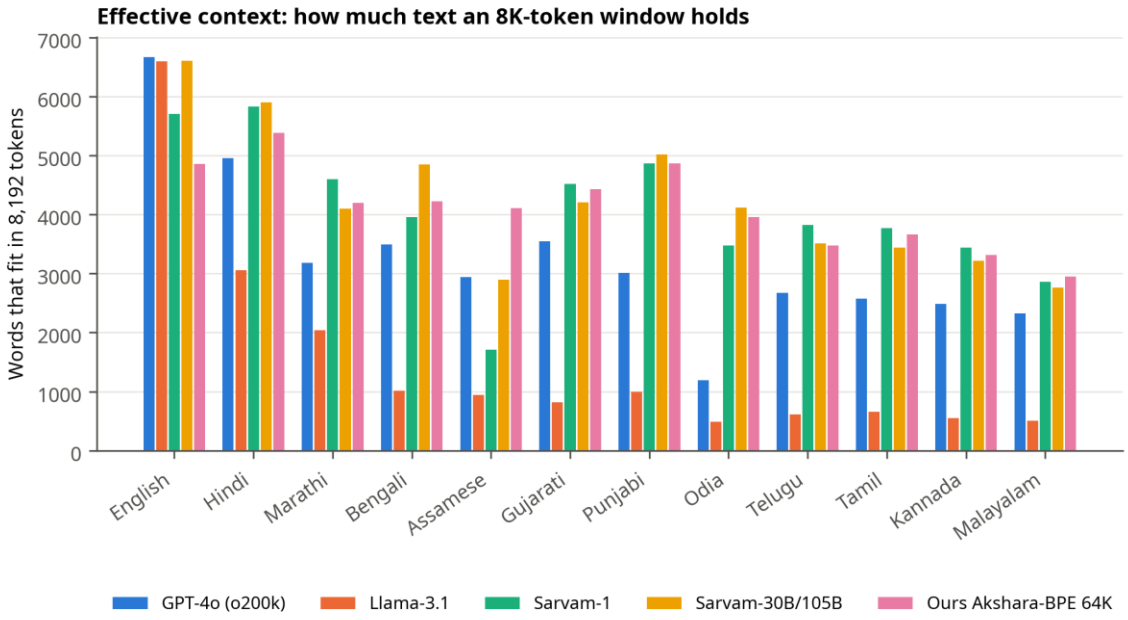


Figure 5: Words that fit in an 8,192-token context window.

5.4 RQ3: Broken aksharas

GPT-4 splits 96.4% of aksharas, Llama-3.1 91.4% and Qwen2.5 84.0%, while GPT-4o splits 29.6%, Sarvam-1 10.0% and Sarvam-30B 9.0%. Sarvam-30B splits Dravidian aksharas 1.40× as often as Indo-Aryan ones; on Telugu, 11.5% against 8.7% for Sarvam-1.

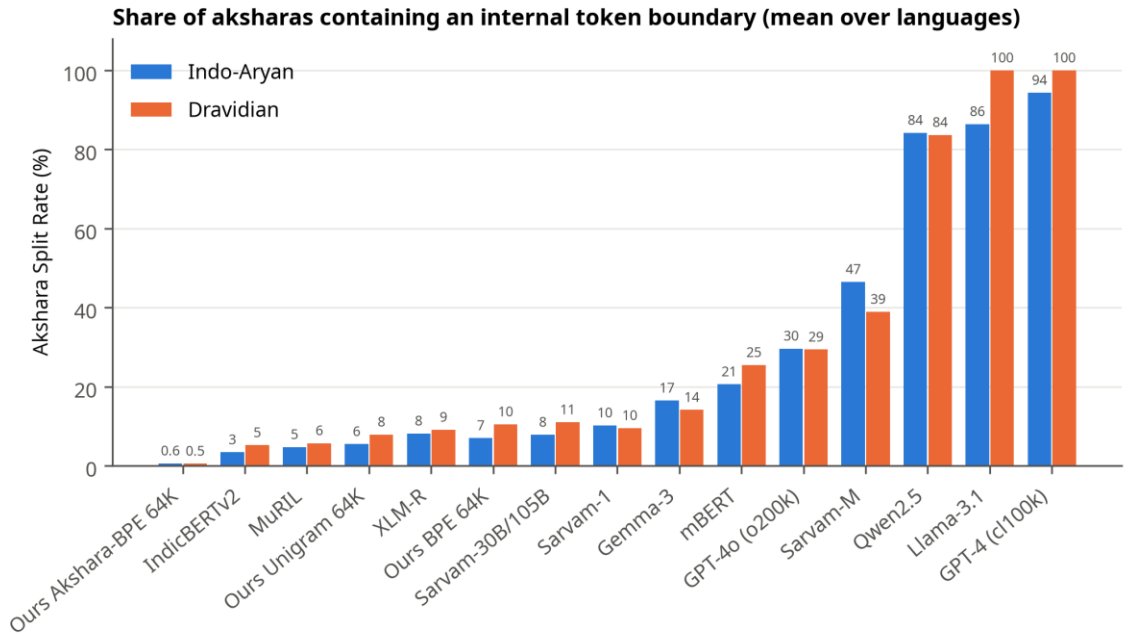


Figure 6: Akshara Split Rate by tokenizer and language family.

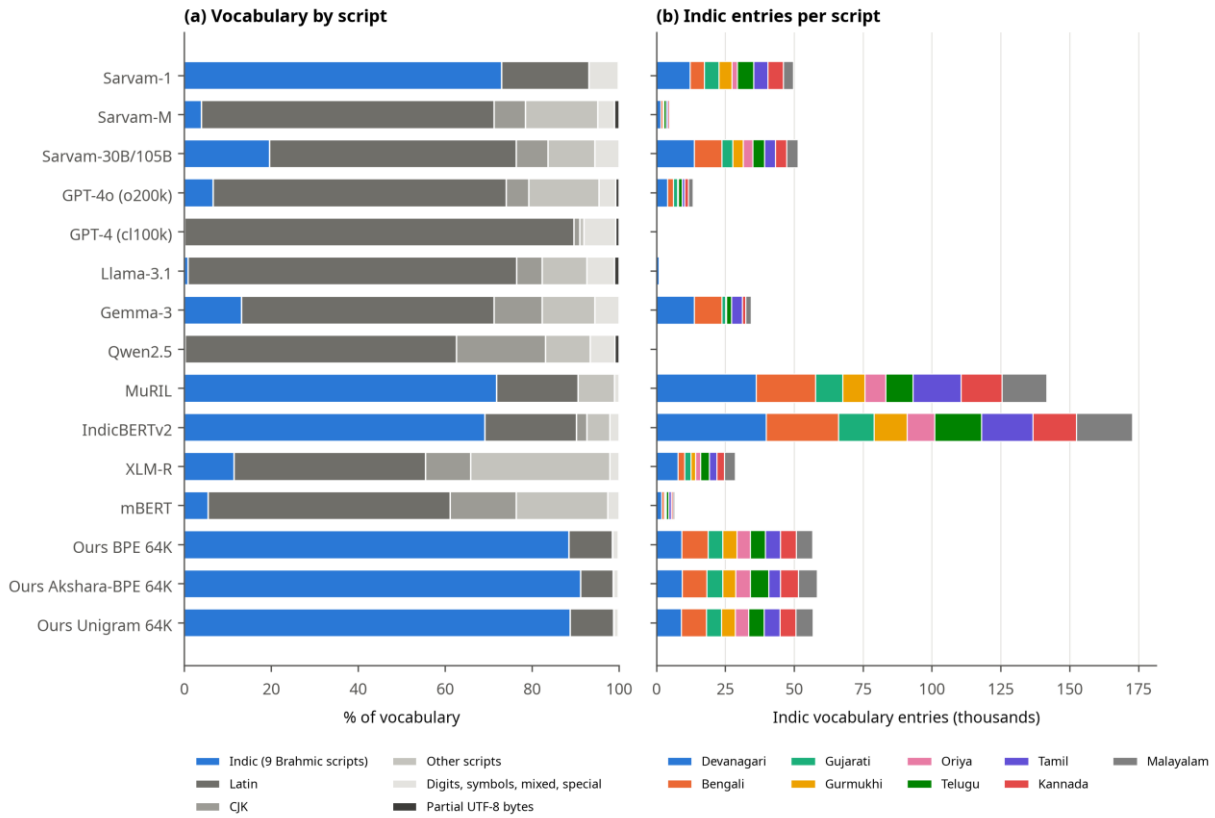


Figure 7: Vocabulary composition by script.

5.5 RQ4: Akshara-atomic BPE

Table 5: Our tokenizers, mean over 11 Indian languages

Model	Fertility	Parity	ASR %	PCW	Mapped
BPE 16K	2.64	1.08	15.5	0.65	–
Akshara-BPE 16K	2.84	1.06	1.6	0.71	8,000
BPE 32K	2.28	1.07	11.5	0.56	–
Akshara-BPE 32K	2.47	1.06	0.7	0.62	16,000
BPE 64K	1.98	1.07	8.3	0.47	–
Akshara-BPE 64K	2.08	1.07	0.6	0.50	22,221
Unigram 64K	1.95	1.07	6.4	0.42	–

Akshara-atomic BPE reduces ASR from 8.3% to 0.6% at 64K and from 15.5% to 1.6% at 16K. Fertility rises by 5.1% and 7.2% respectively. About 20% of the extra tokens at 64K (and 38% at 16K) are standalone word-boundary tokens, which SentencePiece emits when an atomic akshara starts a word. No mapped akshara is ever split; the remaining splits all fall on the 0.59% of FLORES aksharas outside the 64K inventory. Compared with Sarvam-1, the 64K akshara-atomic model has lower fertility (2.08 vs 2.31) and a 17× lower ASR, although its English fertility (1.60) is worse because it was trained on far less English.

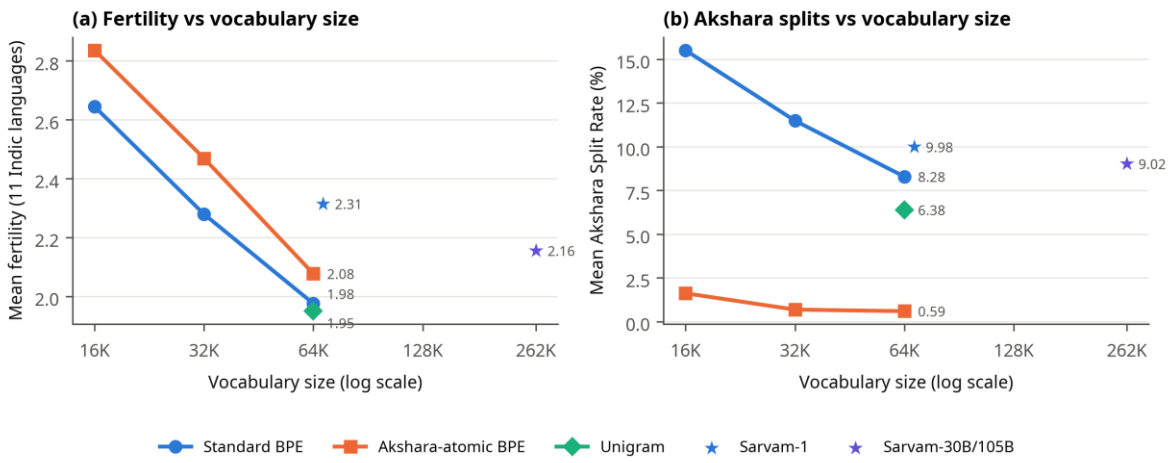


Figure 8: Fertility and ASR against vocabulary size.

5.6 Discussion

For an 8K context, Llama-3.1 fits about 615 Telugu words against 3,821 for Sarvam-1. Reusing a general vocabulary brings strong English and code coverage but inherits its gaps for scripts that receive no new entries. Limitations: whitespace words penalise agglutinative languages, FLORES is one domain, results are intrinsic only, the akshara rules approximate orthographic syllables, and our tokenizers were trained on a small corpus.

Chapter 6: Conclusion and Future Work

Sarvam's tokenizers moved from a dedicated Indic vocabulary (Sarvam-1), to an unchanged general vocabulary (Sarvam-M), to a lightly adapted Gemma-3 vocabulary (Sarvam-30B/105B). The last step helped several Indo-Aryan and low-resource scripts, but made all four Dravidian languages costlier and more often split than under Sarvam-1. Byte-level tokenizers without Indic merges break almost every akshara. Akshara-atomic BPE removes splits of mapped aksharas entirely, at a 5.1–7.2% fertility cost.

Future work: train small language models on each tokenizer to measure downstream effects; add word-initial variants of atomic aksharas to remove standalone boundary tokens; add IN22 and conversational domains; combine akshara-atomic units with morphology-aware segmentation; and extend the analysis to the remaining scheduled languages and scripts.

References

[1] R. Sennrich, B. Haddow, and A. Birch, "Neural machine translation of rare words with subword units," in Proc. 54th Annu. Meeting ACL, 2016, pp. 1715–1725.

[2] T. Kudo, "Subword regularization: Improving neural network translation models with multiple subword candidates," in Proc. 56th Annu. Meeting ACL, 2018, pp. 66–75.

[3] T. Kudo and J. Richardson, "SentencePiece: A simple and language independent subword tokenizer and detokenizer for neural text processing," in Proc. EMNLP: System Demonstrations, 2018, pp. 66–71.

[4] OpenAI, "tiktoken," GitHub repository. [Online]. Available: https://github.com/openai/tiktoken

[5] A. Grattafiori et al., "The Llama 3 herd of models," arXiv:2407.21783, 2024.

[6] P. Rust, J. Pfeiffer, I. Vulić, S. Ruder, and I. Gurevych, "How good is your tokenizer? On the monolingual performance of multilingual language models," in Proc. 59th Annu. Meeting ACL, 2021, pp. 3118–3135.

[7] A. Petrov, E. La Malfa, P. H. S. Torr, and A. Bibi, "Language model tokenizers introduce unfairness between languages," in Advances in Neural Information Processing Systems 36, 2023, pp. 36963–36990.

[8] O. Ahia et al., "Do all languages cost the same? Tokenization in the era of commercial language models," in Proc. EMNLP, 2023, pp. 9904–9923.

[9] V. Zouhar et al., "Tokenization and the noiseless channel," in Proc. 61st Annu. Meeting ACL, 2023, pp. 5184–5207.

[10] D. Kakwani et al., "IndicNLPSuite: Monolingual corpora, evaluation benchmarks and pre-trained multilingual language models for Indian languages," in Findings of EMNLP, 2020, pp. 4948–4961.

[11] S. Doddapaneni et al., "Towards leaving no Indic language behind: Building monolingual corpora, benchmark and models for Indic languages," in Proc. 61st Annu. Meeting ACL, 2023, pp. 12402–12426.

[12] S. Khanuja et al., "MuRIL: Multilingual representations for Indian languages," arXiv:2103.10730, 2021.

[13] J. Gala et al., "IndicTrans2: Towards high-quality and accessible machine translation models for all 22 scheduled Indian languages," Trans. Mach. Learn. Res., 2023.

[14] M. S. U. R. Khan et al., "IndicLLMSuite: A blueprint for creating pre-training and fine-tuning datasets for Indian languages," in Proc. 62nd Annu. Meeting ACL, 2024, pp. 15831–15879.

[15] N. Goyal et al., "The Flores-101 evaluation benchmark for low-resource and multilingual machine translation," Trans. ACL, vol. 10, pp. 522–538, 2022.

[16] NLLB Team et al., "No language left behind: Scaling human-centered machine translation," arXiv:2207.04672, 2022.

[17] M. Brahma et al., “Multilingual tokenization through the lens of Indian languages: Challenges and insights,” arXiv:2506.17789, 2025 (accepted to Findings of ACL 2026).

[18] M. Brahma et al., “MorphTok: Morphologically grounded tokenization for Indian languages,” arXiv:2504.10335, 2025 (TokShop, ICML 2025).

[19] S. Rana et al., “MUTANT: A recipe for multilingual tokenizer design,” arXiv:2511.03237, 2025.

[20] Sarvam AI, “Sarvam 1: the first Indian language LLM,” 24 Oct. 2024. [Online]. Available: https://www.sarvam.ai/blogs/sarvam-1

[21] Sarvam AI, “Sarvam-M multilingual language model,” 23 May 2025. [Online]. Available: https://www.sarvam.ai/blogs/sarvam-m

[22] Sarvam AI, “Open-sourcing Sarvam 30B & 105B models,” 6 Mar. 2026. [Online]. Available: https://www.sarvam.ai/blogs/sarvam-30b-105b

[23] Gemma Team et al., “Gemma 3 technical report,” arXiv:2503.19786, 2025.

[24] Qwen Team et al., “Qwen2.5 technical report,” arXiv:2412.15115, 2024.

[25] Mistral AI, “Mistral NeMo,” 18 Jul. 2024. [Online]. Available: https://mistral.ai/news/mistral-nemo

[26] A. Conneau et al., “Unsupervised cross-lingual representation learning at scale,” in Proc. 58th Annu. Meeting ACL, 2020, pp. 8440–8451.

[27] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, “BERT: Pre-training of deep bidirectional transformers for language understanding,” in Proc. NAACL-HLT, 2019, pp. 4171–4186.

[28] The Unicode Consortium, “Unicode Standard Annex #29: Unicode text segmentation.” [Online]. Available: https://www.unicode.org/reports/tr29/

Appendix A: Sample tokenizations

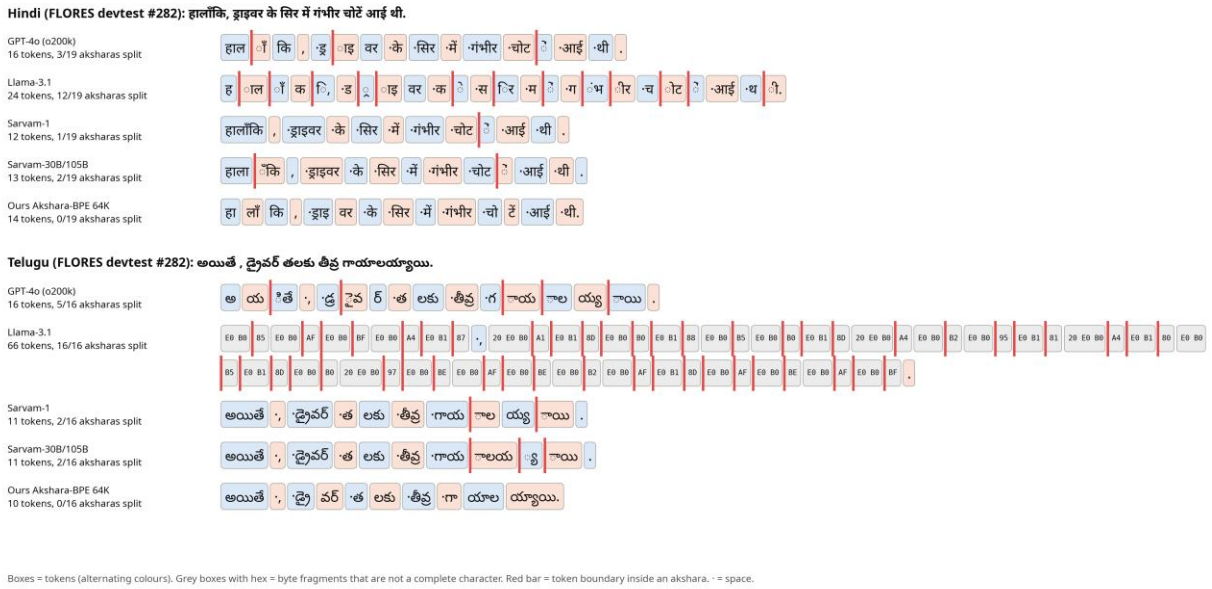


Figure 9: One parallel FLORES sentence in Hindi and Telugu under five tokenizers. Red bars mark token boundaries inside an akshara; grey hex boxes are byte fragments.

Appendix B: Full results

Table 6: Fertility (tokens per word) for every tokenizer and language

Tokenizer	Engl	Hind	Mara	Beng	Assa	Guja	Punj	Odia	Telu	Tami	Kann	Mala
Gemma-3	1.24	1.39	2.00	1.69	2.83	2.42	2.82	4.88	2.85	2.38	3.25	3.36
GPT-4 (cl100k)	1.24	5.06	7.23	8.11	8.71	9.99	8.24	17.29	13.33	12.42	15.00	16.30
GPT-4o (o200k)	1.23	1.65	2.58	2.34	2.78	2.31	2.72	6.87	3.07	3.18	3.30	3.53
IndicBERTv2	1.24	1.24	1.49	1.41	1.60	1.52	1.34	1.61	1.70	1.70	1.85	2.01
Llama-3.1	1.24	2.68	4.01	8.05	8.65	9.99	8.20	16.95	13.33	12.42	14.99	16.29
mBERT	1.32	1.99	2.99	2.79	3.87	3.28	2.44	1.16	3.61	3.63	3.95	5.07
MuRIL	1.26	1.25	1.54	1.44	1.77	1.57	1.44	1.72	1.96	1.75	1.83	2.19
Ours Akshara-BPE 16K	2.31	2.00	2.68	2.63	2.68	2.52	2.19	2.81	3.28	3.15	3.40	3.86
Ours Akshara-BPE 32K	2.02	1.76	2.32	2.28	2.33	2.21	1.95	2.45	2.83	2.72	2.97	3.34
Ours Akshara-BPE 64K	1.69	1.52	1.95	1.94	1.99	1.85	1.68	2.07	2.36	2.24	2.47	2.78
Ours BPE 16K	2.12	1.87	2.48	2.46	2.53	2.37	2.04	2.60	3.04	2.94	3.16	3.59
Ours BPE 32K	1.83	1.64	2.13	2.14	2.20	2.04	1.80	2.25	2.59	2.50	2.69	3.08
Ours BPE 64K	1.60	1.47	1.86	1.87	1.94	1.76	1.62	1.94	2.20	2.15	2.31	2.62
Ours Unigram 64K	1.58	1.45	1.81	1.86	1.94	1.73	1.62	1.92	2.17	2.09	2.27	2.58
Qwen2.5	1.26	4.77	6.71	7.14	7.75	8.93	7.76	13.74	11.43	10.09	11.91	13.39
Sarvam-1	1.44	1.40	1.78	2.07	4.78	1.81	1.68	2.36	2.14	2.18	2.38	2.86
Sarvam-30B/105B	1.24	1.39	2.00	1.69	2.83	1.95	1.63	1.99	2.33	2.38	2.55	2.97
Sarvam-M	1.27	1.95	3.15	2.94	4.76	3.63	3.19	18.36	3.72	3.62	3.83	4.79
XLM-R	1.40	1.49	1.96	2.17	3.01	2.08	1.86	2.26	2.39	2.47	2.58	2.83

Table 7: Akshara Split Rate (%) for every tokenizer and Indian language

Tokenizer	Hind	Mara	Beng	Assa	Guja	Punj	Odia	Telu	Tami	Kann	Mala
Gemma-3	3.6	6.9	6.5	16.6	12.3	25.3	44.2	15.3	6.1	21.0	14.2
GPT-4 (cl100k)	82.9	86.2	93.0	98.2	100.0	100.0	100.0	100.0	100.0	100.0	100.0
GPT-4o (o200k)	12.7	23.3	22.4	26.3	20.9	30.0	72.0	34.7	24.6	28.1	30.2
IndicBERTv2	2.2	4.2	3.3	4.1	4.2	2.4	3.6	4.6	2.3	6.2	7.8
Llama-3.1	58.4	62.3	91.1	96.6	100.0	98.2	98.3	100.0	100.0	100.0	100.0
mBERT	17.1	23.8	21.6	31.7	29.2	20.8	0.2	28.3	12.4	29.3	32.0
MuRIL	2.5	5.4	2.9	5.1	5.5	5.2	6.6	8.9	1.5	6.0	6.5

Ours Akshara-BPE 16K	1.4	1.0	2.1	2.0	1.3	2.1	2.0	1.8	0.0	2.0	2.0
Ours Akshara-BPE 32K	0.4	0.2	0.5	0.4	0.2	1.9	1.2	0.7	0.0	1.1	0.8
Ours Akshara-BPE 64K	0.3	0.1	0.4	0.2	0.1	1.9	1.1	0.5	0.0	1.0	0.7
Ours BPE 16K	12.1	13.4	19.9	16.4	11.1	11.0	11.7	17.4	10.2	22.3	25.1
Ours BPE 32K	8.7	9.8	15.0	12.2	7.7	8.1	8.2	12.6	7.3	16.8	19.9
Ours BPE 64K	6.0	7.1	10.9	9.1	4.8	5.8	5.4	9.0	5.3	12.6	15.1
Ours Unigram 64K	5.2	7.2	5.9	4.7	5.4	5.8	4.3	8.7	3.2	8.2	11.7
Qwen2.5	75.9	80.7	79.7	82.8	86.6	86.6	97.2	88.4	89.4	78.5	78.6
Sarvam-1	3.5	5.1	10.7	33.0	4.8	6.2	8.2	8.7	5.3	10.9	13.3
Sarvam-30B/105B	3.6	6.9	6.5	16.6	7.7	7.4	6.4	11.5	6.1	14.1	12.4
Sarvam-M	19.4	34.5	32.2	51.6	47.8	41.9	98.3	45.2	30.5	35.4	44.8
XLM-R	5.3	8.0	8.7	11.5	8.7	8.0	7.1	10.5	3.5	10.4	12.2

Language abbreviations use the first four letters of each language name. The complete long-format results, including parity, PCW, intra-character splits, Rényi efficiency and confidence intervals, are in results/benchmark_long.csv and results/benchmark_summary.csv in the repository (https://github.com/jkknikhil-git/indic-tokenizer-study).

Code and data: https://github.com/jkknikhil-git/indic-tokenizer-study